A Multithreaded Real-Time Collector for the Bluesky Jetstream Firehose

Architecture, Synchronisation and Verification

Evangelos Moschou
Embedded Systems — Final Assignment
https://github.com/EvangelosMoschou/Embedded_assignment

September 2026

1 System Overview

The system is a self-contained acquisition node that runs unattended for 24 h on a Raspberry Pi Zero W — a *single-core* ARM1176 (ARMv6, 32-bit) at 1 GHz with 512 MB of RAM and no real-time kernel. It subscribes to the public Bluesky Jetstream firehose over a WebSocket (`wss://jetstream1.us-east.bsky.network/subscribe?wantedCollections=app.bsky.feed.post`), classifies every incoming JSON message by its top-level `kind` field (`commit`, `identity`, `account` or `info`), and appends one line of telemetry per second. Traffic is non-uniform — 39.5 messages per second on average with peaks above 340 — so three activities with incompatible timing requirements must coexist: ingestion is bursty and must never block, classification is throughput-oriented, and telemetry is strictly periodic. They are placed on separate threads and decoupled by a bounded buffer, so a delay in one cannot propagate into the others; the goals are *low jitter*, *zero data loss* and *bounded memory* rather than hard-deadline guarantees.

2 Architecture

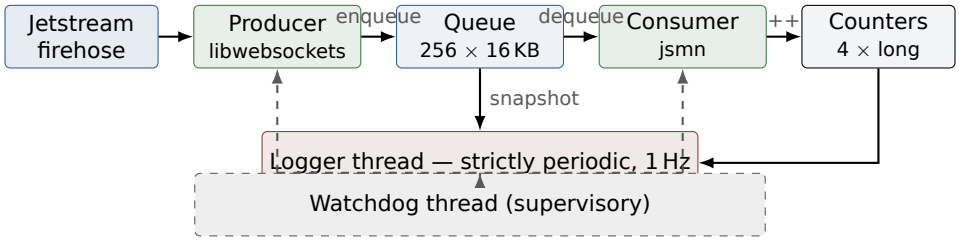


Figure 1: Data flow. The producer and the logger touch the queue; the consumer and the logger touch the counters; the logger is the only thread performing I/O; the watchdog only observes.

Producer (asynchronous, network-facing). It owns a single libwebsockets client context and runs the event loop `lws_service(ctx, 50)`, performing no downstream processing: a complete message is enqueued, the consumer is signalled, and control returns to the network immediately, so socket buffers are drained as fast as possible. Two details are essential. First, *reassembly*: libwebsockets delivers anything larger than its receive buffer as several callbacks, so fragments are accumulated in a 16 KB staging buffer and published only when `lws_is_final_fragment()` reports the end of the frame — otherwise one message would be parsed as several invalid ones. Second, *bounded insertion*: if the queue is full the frame is dropped and counted rather than blocking the network thread, because blocking there would lose data at a layer the application cannot observe. On failure the producer reconnects with exponential backoff (1, 2, 4, …, 60 s), reset after any connection that lasted longer than a minute.

Consumer (event-driven, no I/O). It sleeps on the queue's condition variable and, for each message, releases the slot, parses the JSON with **jsmn** — a zero-allocation tokeniser (256 tokens, `JSMN_PARENT_LINKS`) whose cost is independent of message size — and increments the corresponding counter under its own mutex. Per the specification this thread performs no screen output and no file I/O. Classification is made robust by falling back to the tokens parsed so far whenever the parser reports an error, which happens when a frame exceeds the 16 KB slot; since `kind` appears near the beginning of a Jetstream message, such frames are still counted correctly instead of discarded.

Logger (strictly periodic). It is the only thread with side effects on storage and the component whose timing is measured. It fires exactly once per second from `clock_nanosleep(CLOCK_MONOTONIC, TIMER_ABSTIME, &next, NULL)`, where `next` is advanced by one second from the *previous ideal deadline* rather than from the actual wake-up. Each activation it locks the counter mutex briefly to read and reset the four counters, samples the queue occupancy, computes CPU utilisation from `/proc/stat`, and appends a `CLOCK_REALTIME` timestamp. The absolute-deadline scheme is what prevents drift from accumulating: a late wake-up shortens the next interval instead of shifting every subsequent sample, so drift is zero by construction and any residual deviation of the `Seconds` column is attributable to the wall clock, not the scheduler. Sampling once per second also bounds the observer effect on the other threads to one short critical section per second, regardless of the message rate.

Watchdog (supervisory). A fourth, non-essential thread observes the liveness of the three workers through atomic heartbeats; if a worker reports no progress for 5 s the process exits with a non-zero status, which the `systemd` unit (`Restart=always`) turns into a fresh process. It also emits the `WATCHDOG=1` notification required by `WatchdogSec`, and takes part in no data path — it only ever reads shared state.

3 Synchronisation and Race Conditions

Two mutexes protect the two shared structures, and no lock is ever held across a blocking call:

Shared state	Protected by	Written by	Read by
Queue (head, tail, count, peak)	`fq.mut`	producer	consumer, logger
Message counters	`counters_mut`	consumer	logger

The queue is a classical bounded-buffer of **256 slots of 16 KB each**, statically allocated (4 MB), with *two* condition variables: the producer waits on `notFull` and the consumer on `notEmpty`. Deadlock is avoided because each condition variable is signalled only while holding the queue mutex, and because the consumer releases the slot *before* parsing, so the slow part never blocks the producer. Three further details resolve specific races: (i) the consumer's wait is a *one-second timed wait* that refreshes its heartbeat on each timeout — a plain `pthread_cond_wait()` would make an idle consumer, for instance during a 60 s reconnect backoff, indistinguishable from a dead one; (ii) the termination handler performs a single atomic store and nothing else, since `pthread_cond_broadcast()` is not async-signal-safe, and the other threads notice the flag within one second; (iii) the shared heartbeats and the shutdown flag are C11 atomics of **4-byte** width, a deliberate choice — on ARMv6 8-byte atomics are not lock-free and would require linking against `libatomic` (Section 5).

4 Log File

Telemetry is appended to a single file, `metrics_log.txt`, with exactly the prescribed eight comma-separated fields — `Seconds` and `Nanoseconds` from `clock_gettime(CLOCK_REALTIME)`, the four per-second message counters (reset after each sample), the queue occupancy at the instant of logging, and the `/proc/stat` CPU utilisation over the preceding second. The header is written only when the file is created, so a restart continues the same dataset instead of injecting a header line into the middle of it.

5 Off-Target Verification

Because the Pi was intended to run unattended for 24 h, the code was first exercised entirely off-target: a **mock firehose server** with an exactly known message distribution checks the logged counters against ground truth, three **sanitiser builds** verify bounds, leaks and locking, an **ARMv6 cross-compile** probe confirms that the target build keeps the lock-free guarantee, and a **leak probe** repeats the time-driven operations 86 400 times. It was not merely confirmatory: it found four defects that would have corrupted the dataset, including misclassification of fragmented messages and false watchdog termination during network outages. After correction the suite reports exact counter agreement and no memory growth.

6 Measured Results

Measured on the deployed system over a continuous 24-hour window (20 September 2026, 00:00:00–23:59:59 local time):

Quantity	**Value**	**Notes**
Samples logged	86 394 of 86 400	99.993 % coverage
Message rate	39.5 Hz mean, 343 Hz peak	peak-to-mean $\approx 8.7\times$
Jitter, median	104 µs	95th pct. 362 µs
Jitter, 99th pct.	845 µs	worst case 18.1 ms
Queue occupancy	13.7 % peak	35 of 256 slots; 0 frames dropped
CPU, whole process	7.8 % mean	of one core
Resident memory	12.2 MB	of which 4 MB pre-allocated queue

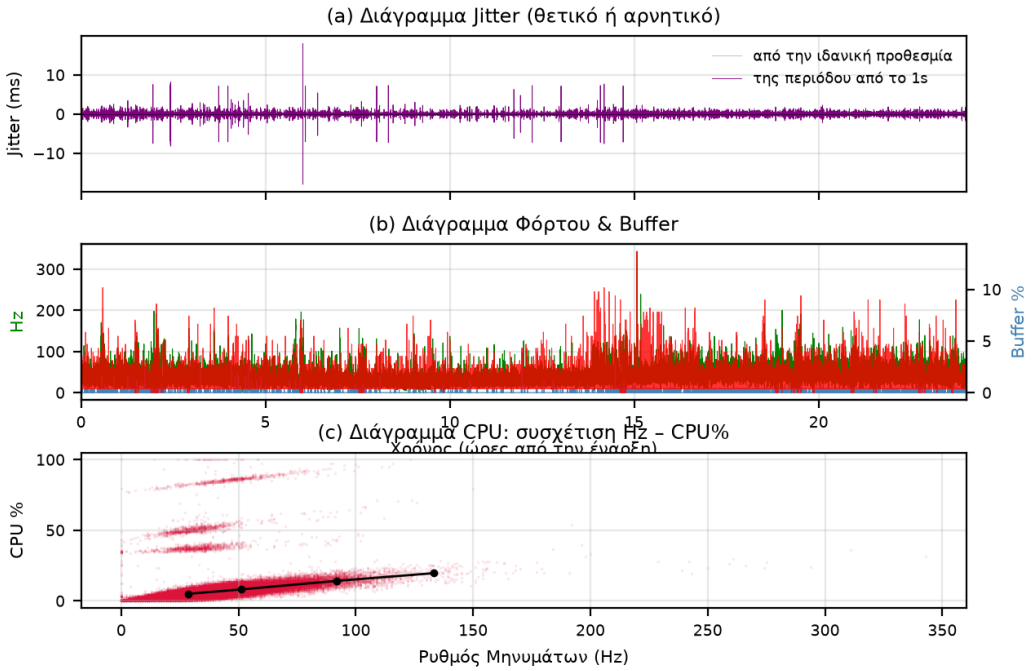


Figure 2: The three required plots over the full 24 hours. **(a)** *Jitter*: deviation from the absolute deadline (grey, ≥ 0 by construction) and the signed deviation of each one-second period from the ideal (dark). **(b)** *Load and buffer*: message rate vs. buffer occupancy. **(c)** *CPU*: the correlation of message rate (abscissa) with CPU utilisation, with the median of each rate band in black.

Three observations follow. CPU rises monotonically with the incoming rate, from about 5 % below 40 Hz to about 20 % near 150 Hz, yet stays far below one core; the rest of the cost is TLS framing. And the queue occupancy is best read as *headroom* — it scales with the rate while never saturating, so the drop counter stayed at zero.

7 Long-Run Behaviour and Network Resilience

Zero loss. The drop counter stayed at zero for the whole window: every frame the network delivered was enqueued and classified. Frames larger than the 16 KB slot (well below 0.02 % of the total) were still classified correctly and are counted separately as truncated.

One recovered stall. At 14:43:38 the producer stopped renewing its heartbeat while consumer and logger continued normally (the watchdog logged `p=7 c=1 l=1`, and the logger's jitter stayed near 100 µs). No disconnection, storage error, kernel message or thermal event accompanied it, so the blocker lay inside the network library with the connection established. The watchdog terminated the process after 5 s, `systemd` restarted it 5.5 s later, and the socket returned within a further second: **6 seconds out of 86 400** (0.007 %). At 01:28:17 the log shows an *identical* seven-second interval with no messages and *no* restart, since the producer was alive and only the server was quiet — so the criterion is not "absence of data" but "absence of progress by the thread".

Flapping is absorbed. 42 disconnections and 42 reconnections occurred during the day, never more than seven in a single hour. Each produced an automatic reconnection with exponential backoff; none restarted the process, lost a classified message, or produced a segmentation fault.

Memory is bounded. Resident memory grew by only 224 kB across the whole day, from 11.9 to 12.2 MB, the rate decaying from about 128 kB per hour initially to about 21 kB per hour by hours eight to twelve — the signature of a statically allocated working set reaching its high-water mark, not a leak (`malloc_stats()`: 1.43 MB reserved).